

Машинное обучение (Machine Learning)

Обучение с подкреплением (Reinforcement Learning)

Уткин Л.В.

Санкт-Петербургский политехнический университет Петра Великого

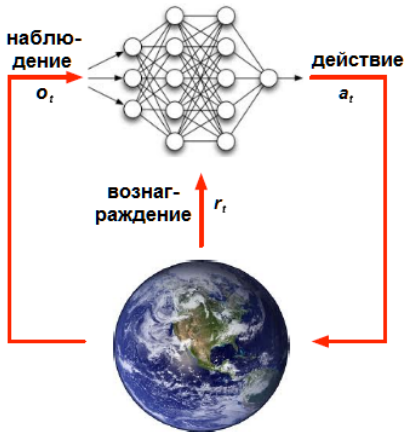


- 1 Постановка задачи
- 2 Марковский процесс принятия решений (MDP)
- 3 Многорукий бандит
- 4 Алгоритмы обучения

Постановка задачи

- В отличие от стандартных задач обучения с учителем и без учителя, появляется новый “субъект” - агент.
- Параметры задачи обучения могут изменяться в процессе решения при получении информации, насколько то или иное действие (решение, параметр) правильное или нет.
- Для этого агент взаимодействует с окружающей средой, предпринимая действия; окружающая среда его поощряет за эти действия или, наоборот, наказывает, а агент продолжает предпринимать эти же или другие действия.





Постановка задачи

- На каждом шаге t агент:
 - Выполняет действие a_t
 - Получает наблюдение o_t
 - Получает вознаграждение r_t
- Среда:
 - Получает действие a_t
 - Выдает наблюдение o_{t+1}
 - Выдает вознаграждение r_{t+1}

Постановка задачи

- Опыт (история) - последовательность наблюдений, действий, вознаграждений

$$H_t = (o_1, r_1, a_1, o_2, r_2, a_2, \dots, o_t, r_t)$$

- То, что случится далее, зависит от истории: - агент выберет действие, среда сгенерирует наблюдение и вознаграждение
- **Состояние** - функция опыта для определения, что произойдет далее

$$s_t = f(H_t)$$

- В полностью наблюдаемой среде

$$s_t = f(o_t)$$

Состояние системы

- Состояние системы - параметр или множество параметров, используемых для описания системы.
- Например, географические координаты робота - **состояние**.
- Если состояние системы изменяется во времени - динамическая система (двигающийся робот)
- Очередь в магазин: **состояние** - число людей - динамическая система
- Переход из состояния в состояние обычно случайный
- Время перехода = 1

Действие

- Движение робота управляется. Предположим робот может двигаться дискретными шагами на север, юг, восток и запад.
- Эти четыре направления - **действия**.
- Для очереди в кассу: когда число покупателей превышает заданное число (пусть 10), оставшиеся покупатели перенаправляются в другую кассу. Два **действия**: 1 - перенаправлять, 2 - не перенаправлять.

- *AlphaGo*: $+r$ - за победу в партии, $-r$ - за проигрыш
- *Atari*: $+r$ - за увеличение счета, $-r$ - за уменьшение счета
- *Робот*: $+r$ - за прямой шаг, $-r$ - за падение
- *Инвестиции*: $+r$ - за каждый заработанный рубль, $-r$ - за каждый потерянный рубль
- *Вертолет*: $+r$ - за следование намеченной траектории, $-r$ - за отклонение от нее

- Стратегия (policy) π : определяет действие, которое должно быть выбрано в каждом состоянии (отображение из состояния в действие), т.е. это функция поведения агента
- Функция полезности (value function) - оценка того, насколько полезно состояние
- Модель (model) - представление агента о среде

Стратегия

- Стратегия (policy) π : определяет действие, которое должно быть выбрано в каждом состоянии (отображение из состояния в действие), т.е. это функция поведения агента
 - детерминированная стратегия: $a = f(s)$
 - вероятностная стратегия:
$$\pi(a | s) = \Pr\{a_t = a | s_t = s\}$$

- Без обратной связи, указывающей, какое действие является хорошим, а какое плохим, агент не сможет принимать решения, что делать дальше.
- Агент должен знать, что его выигрыш - это благоприятный исход, а проигрыш - неблагоприятный.
- Обратная связь называется **подкреплением**.
- Получаем **обучение с подкреплением** (reinforcement learning).

Задача обучения с подкреплением состоит в том, чтобы обеспечить использование наблюдаемых вознаграждений для определения в процессе обучения оптимальной (или почти оптимальной) стратегии для данной среды.

- На каждом шаге агент может находиться в состоянии $s \in S$.
- На каждом шаге агент выбирает из имеющегося набора действий некоторое действие $a \in A$.
- Окружающая среда сообщает агенту, какую награду r он за это получил и в каком состоянии s' после этого оказался.

- Пассивный агент просто наблюдает за происходящим миром и пытается узнать полезность нахождения в различных состояниях
- Или: пассивный агент имеет **фиксированную стратегию π**
- Служит в качестве компонента активных алгоритмов обучения

- Агент изменяет свою стратегию в процессе обучения
- Агент пытается найти **оптимальную** (или, по крайней мере, **хорошую**) стратегию
- Аналогично решению, лежащему в основе процесса принятия решений Маркова

Функция полезности

- Знаем уже: действие, состояние, вознаграждение, стратегия
- **Value function, функция полезности $V^\pi(s)$** - предсказание будущего вознаграждения, используется для оценки состояния или ожидаемый долгосрочный доход с дисконтом
- По функции полезности происходит отбор действий

$$V^\pi(s) = \mathbb{E}_\pi [r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \mid s_t = s]$$

Модель

- Модель служит для предсказания того, что произойдет в среде в следующий момент времени
- Например
 - *модель переходов* $\mathcal{P}$ предсказывает следующее состояние

$$\mathcal{P}_{s,s'}^a = \Pr\{s_{t+1} = s' \mid s_t = s, a_t = a\}$$

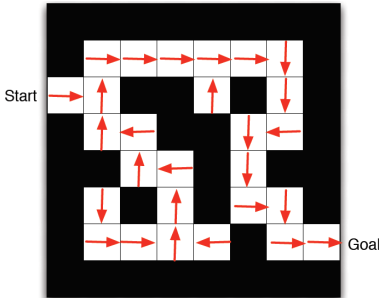
- *модель вознаграждений* $\mathcal{R}$ предсказывает следующее вознаграждение

$$\mathcal{R}_s^a = \mathbb{E}\{r_t \mid s_t = s, a_t = a\}$$

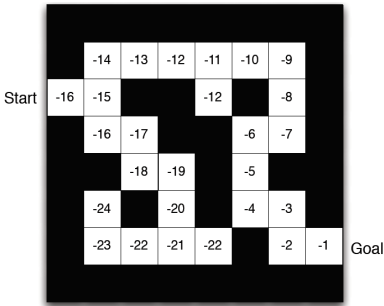


Goal

- Вознаграждение: -1 за каждый шаг
- Действия: $N \uparrow, E \rightarrow, S \downarrow, W \leftarrow$
- Состояния: местоположение агента (номер клетки)
- Найти кратчайший путь (чем меньше шагов, тем больше сумма вознаграждений)

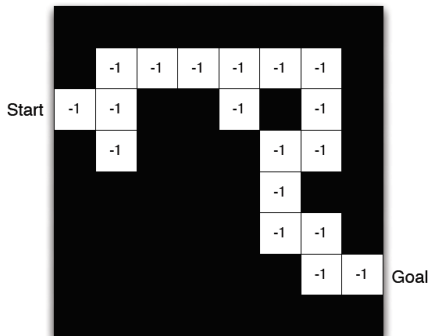


- Стрелки представляют собой стратегию $\pi(s)$ для каждого состояния s



- Числа - значение $V^\pi(s)$ для каждого состояния

Пример



- Макет сетки - модели переходов
- Числа - немедленное вознаграждение для каждого состояния - следующее вознаграждение $\mathcal{R}_s^a$ для каждого состояния s
- Клетки отображают модель переходов $\mathcal{P}_{s s'}^a$



591. TOKOM (=100)

- После небольшого исследования мышь может найти “оазис” (локальный оптимум) с 3 источниками воды возле входа и оставаться там, постоянно получая небольшие вознагражд. этих источников и никогда не отправляясь дальше в лабиринт, чтобы искать более крупный приз.
- Но мышь не достигнет лучший “оазис” или сыр!
- Компромисс между разведкой и эксплуатацией (дилемма разведка-эксплуатация).

Мышь в лабиринте

- Простая стратегия - мышь принимает самое известное действие большую часть времени (скажем, 80% времени), но иногда изучает новое, случайно выбранное направление, даже если оно уходит от известного вознаграждения.
- Эта стратегия - ϵ -жадная, где ϵ - это процент времени, когда агент принимает случайное действие.
- Со временем ϵ может уменьшаться (разведываем все меньше, так как уже что-то знаем)



Многорукий бандит - более формально

- Многорукий бандит - кортеж $\langle A, R \rangle$
- Состояние одно $|S| = 1$.
- A - известное множество m действий (или "рук")
- $p^a(r) = \Pr[r \mid a]$ - неизвестное распределение вероятностей вознаграждений за действие $a \in A$
- На каждом шаге t агент выбирает действие $a_t \in A$ с учетом стратегии $a_t \sim \pi_t(a)$
- Среда генерирует вознаграждение $r_t \sim p^{a_t}(r)$
- Цель - максимизировать суммарное вознаграждение $\sum_{i=1}^t r_i$

Многорукий бандит - жадный алгоритм (1)

- Ожидаемое (среднее) значение вознаграждения $Q(a)$ (функция полезности действия)
- Ценность действия a : $Q^*(a) = \lim_{t \rightarrow \infty} Q_t(a) \rightarrow \max_{a \in A}$
- Агент вычисляет оценки $Q_t(a)$
- Стратегия, максимизирующая текущую оценку ценности $A_t \subseteq A$: $A_t = \arg \max_{a \in A} Q_t(a)$ (дергаем ручку с наибольшим средним)

Многорукий бандит - жадный алгоритм

- Жадная стратегия выбирать любое действие из A_t :

$$\pi_{t+1}(a) = \frac{1}{|A_t|} [a \in A_t]$$

- Или более просто

$$Q_t(a) = \frac{r_1(a) + r_2(a) + \dots + r_t(a)}{t}$$

- $r_N(a)$ - число, когда выбирается действие a за первые t шагов, $r_i(a)$ - полученные в результате подкрепления.

Проблемы и их решение (1)

- Если на первом шаге автомат дал 0, то к нему уже не будет обращение

$$Q_t(a) = \frac{r_1(a) + r_2(a) + \dots + r_t(a)}{t}$$

- Полезная эвристика - оптимизм при неопределенности
 - ϵ -жадная стратегия - с вероятностью ϵ дергаем случайную ручку - можем дернуть за любую ручку

$$\pi_t(a) = \frac{1 - \varepsilon}{|A_t|} [a \in A_t] + \frac{\varepsilon}{|A|}$$

- доверительный интервал для средних (из распредел. Стьюдента)

$$\left(\bar{x} - 1.96 \frac{\sigma}{\sqrt{N}}; \bar{x} + 1.96 \frac{\sigma}{\sqrt{N}} \right)$$

используем верхнюю границу, интервал сужается с n

Проблемы и их решение (2)

- Выбор действия с максимальной верхней оценкой ценности:

$$A_t = \arg \max_{a \in A} \left(Q_t(a) + \delta \sqrt{\frac{2 \ln t}{k_t(a)}} \right)$$

где $k_t(a) = \sum_{i=1}^t [a_i = a]$; δ - параметр компромисса

- Чем меньше $k_t(a)$, тем менее исследована стратегия, тем выше должна быть вероятность выбора a
- Чем больше δ , тем стратегия более исследовательская

Исследование по Больцману

- Исследование по Больцману (Boltzmann exploration):

$$\pi_t(a) = \frac{\exp(Q_t(a)/T)}{\sum_{a'} \exp(Q_t(a')/T)}$$

- Увеличивая температуру, уменьшаем разницу между вероятностями выбора
- Уменьшая температуру, приближаем выбор к жадному
- Обычно температура понижается со временем

Правило обновления

- Как пересчитать Q_t при поступлении новой информации?

$$\begin{aligned}Q_{t+1}(a) &= \frac{1}{t+1} \sum_{i=1}^{t+1} r_i = \frac{1}{t+1} \left(r_{t+1} + \sum_{i=1}^t r_i \right) \\&= \frac{1}{t+1} (r_{t+1} + tQ_t(a) + Q_t(a) - Q_t(a)) \\&= \frac{1}{t+1} (r_{t+1} + (t+1)Q_t(a) - Q_t(a)) \\&= Q_t(a) + \frac{1}{t+1} (r_{t+1} - Q_t(a))\end{aligned}$$

- Не нужно хранить всю историю, только текущий шаг и число экспериментов

Многорукий бандит - жадный алгоритм

$$Q_{t+1}(a) = Q_t(a) + \frac{1}{t+1} (r_{t+1} - Q_t(a))$$

- $Q_{t+1}(a)$ - среднее вознаграждение за $t + 1$ шагов
- $\frac{1}{t+1}$ - скорость обучения или шаг
- r_{t+1} - вознаграждение на шаге $t + 1$
- Шаг $\frac{1}{t+1}$ изменяется, но им можно управлять, чтобы улучшить процедуру обучения

Вычисление оценок

Сдвигаем оценку так, чтобы уменьшилась ошибка

$$\text{Новая оценка} = \text{Старая оценка} + \text{шаг} * (\text{Цель} - \text{Старая оценка})$$

(Цель - Старая оценка) - ошибка

Постоянная скорость обучения

- Пусть

$$Q_{t+1} = Q_t + \alpha (r_{t+1} - Q_t)$$

- Веса затухают экспоненциально

$$\begin{aligned} Q_t &= Q_{t-1} + \alpha (r_t - Q_{t-1}) = \alpha r_t + (1 - \alpha) Q_{t-1} \\ &= \alpha r_t + (1 - \alpha) \alpha r_{t-1} + (1 - \alpha)^2 Q_{t-2} \\ &= (1 - \alpha)^t Q_0 + \sum_{i=1}^t \alpha (1 - \alpha)^{t-i} r_i \end{aligned}$$

- При $\alpha = \text{const}$ - экспоненциальное скользящее среднее.

Экспоненциальное скользящее среднее (1)

- Задача прогнозирования временного ряда $y_0, \dots, y_t, \dots$
- простейшая регрессионная модель $y_t = c$
- наблюдения с весами убывающими в прошлое:
прогноз $\hat{y}_{t+1}$ методом наименьших квадратов:

$$\sum_{i=0}^t w_{t-i} (y_i - c)^2 \rightarrow \min_c, \quad w_i = \phi^i, \quad \phi \in (0, 1).$$

- Аналитическое решение - (регрессия Надарая-Уотсона):

$$c = \hat{y}_{t+1} = \frac{\sum_{i=0}^t \phi^i y_i}{\sum_{i=0}^t \phi^i}.$$

- Аналогично $\hat{y}_t$: оценим $\sum_{i=0}^t \phi^i \approx \sum_{i=0}^{\infty} \phi^i = \frac{1}{1-\phi}$, тогда

$$\hat{y}_{t+1} = \hat{y}_t \phi + (1 - \phi)y_t.$$

Экспоненциальное скользящее среднее (2)

- Регрессия Надарая-Уотсона:

$$c = \hat{y}_{t+1} = \frac{\sum_{i=0}^t \phi^i y_i}{\sum_{i=0}^t \phi^i}.$$

- Аналогично $\hat{y}_t$: оценим $\sum_{i=0}^t \phi^i \approx \sum_{i=0}^{\infty} \phi^i = \frac{1}{1-\phi}$, тогда

$$\hat{y}_{t+1} = \hat{y}_t \phi + (1 - \phi)y_t.$$

- Заменяв $1 - \phi = \alpha$, получим

$$\hat{y}_{t+1} = \hat{y}_t(1 - \alpha) + \alpha y_t = \hat{y}_t + \alpha(y_t - \hat{y}_t)$$

Метод сравнения с подкреплением (1)

- Интуиция: мы ищем «большие» вознаграждения. А что такое «большие»?
- Можно сравнивать со средним вознаграждением по всем ручкам; это называется метод сравнения подкреплений (reinforcement comparison).
- В таких методах обычно нет action values Q_k , есть предпочтения $p_t(a)$
- вероятности $\pi_{t+1}(a)$ можно получить, например, по Больцману:

$$\pi_t(a) = \frac{\exp(p_t(a))}{\sum_{b \in A} \exp(p_t(b))}$$

Метод сравнения с подкреплением (2)

- Идея: использовать в softmax не сами значения премий, а их разности со средней премией; на каждом шаге обновляем среднее вознаграждение и предпочтения
- $\bar{r}_{t+1} = \bar{r}_t + \alpha(r_t - \bar{r}_t)$ - средняя премия по действиям
- $p_{t+1}(a_t) = p_t(a_t) + \phi(r_t - \bar{r}_t)$ - предпочтения действий (усредняемая разность)
- Когда разность > 0 , $\exp()$ большая, иначе малая
- $\pi_{t+1}(a) = \frac{\exp(p_{t+1}(a))}{\sum_{b \in A} \exp(p_{t+1}(b))}$ - softmax-страт. агента
- Замена $1 - \phi = \alpha$:
 $\hat{y}_{t+1} = \hat{y}_t(1 - \alpha) + \alpha y_t = \hat{y}_t + \alpha(y_t - \hat{y}_t)$

Метод преследования жадной стратегии

- Вместо собственно жадной стратегии

$$\pi_{t+1}(a) = \frac{1}{|A_t|} [a \in A_t]$$

предлагается *преследование* (сглаживание) жадной стратегии:

$$\pi_{t+1}(a) = \pi_t(a) + \phi \left(\frac{1}{|A_t|} [a \in A_t] - \pi_t(a) \right)$$

- Текущее распределение $\pi_t(a)$ смешивается с жадной стратегией
- $\pi_0(a)$ можно взять равномерным

Сожаления

- Функция $Q(a)$ - среднее вознаграждение для действия a : $Q(a) = E[r \mid a]$
- Оптимальное значение V^* :

$$V^* = Q(a^*) = \max_{a \in A} Q(a)$$

- **Сожаление** - возможные потерь за один шаг

$$I_t = E[V^* - Q(a_t)]$$

- Суммарные сожаления -

$$L_t = E \left[\sum_{i=1}^t V^* - Q(a_i) \right]$$

- Максимум суммарного вознаграждения = минимум суммарного сожаления

Подсчет сожалений

- $N_t(a)$ - среднее число выбора действия a
- Зазор Δ_a - разность между значений действия a и оптимального действия a^* : $\Delta_a = V^* - Q(a)$
- Сожаление - функция зазора и $N_t(a)$:

$$\begin{aligned} L_t &= E \left[\sum_{i=1}^t V^* - Q(a_i) \right] \\ &= \sum_{a \in A} E[N_t(a)] (V^* - Q(a)) = \sum_{a \in A} E[N_t(a)] \Delta_a \end{aligned}$$

- Хороший алгоритм обеспечивает малое значение $N_t(a)$ для больших зазоров
- Проблема: зазоры не известны!

Мышь в лабиринте - марковский процесс решений (MDP)

- Мышь, блуждающая по лабиринту, может быть формализована как процесс принятия решений Маркова (для которого указаны вероятности перехода из состояния в состояние)
- MDP включает: конечное множество состояний, набор действий в состоянии, переходы между состояниями, вознаграждения, коэффициент дисконтирования, отсутствие памяти
- Многие задачи RL могут быть сведены к задаче MDP

Марковское состояние

- Марковское состояние содержит всю необходимую информацию, которая может быть иметься в истории
- Состояние s_t называется марковским, если и только если

$$P\{s_{t+1} \mid s_t\} = P\{s_{t+1} \mid s_1, \dots, s_t\}$$

- Будущее не зависит от прошлого и определяется только настоящим

$$H_{1:t} \rightarrow s_t \rightarrow H_{t+1:\infty}$$

- Если известно текущее состояние, история может быть отброшена
- Текущее состояние содержит всю необходимую статистику о будущем

Матрица переходов

- Вероятность перехода для марковского состояния s в следующее состояние s' определяется как

$$\mathcal{P}_{s,s'} = \Pr\{s_{t+1} = s' \mid s_t = s\}$$

- Матрица переходов $\mathcal{P}$ определяет вероятности переходов между всеми возможными состояниями:

$$\mathcal{P} = \begin{bmatrix} \mathcal{P}_{11} & \cdots & \mathcal{P}_{1n} \\ \cdots & \cdots & \cdots \\ \mathcal{P}_{n1} & \cdots & \mathcal{P}_{nn} \end{bmatrix}$$

- Каждая строка в сумме равна 1

Марковский процесс

- Марковский процесс - это случайный процесс без памяти, т.е. последовательность случайных состояний $s_1, s_2, \dots$
- Марковский процесс - это пара $\langle \mathcal{S}, \mathcal{P} \rangle$, где $\mathcal{S}$ - конечное множество состояний, $\mathcal{P}$ - матрица переходов

$$\mathcal{P}_{s,s'} = \Pr\{s_{t+1} = s' \mid s_t = s\}$$

Полная постановка задачи обучения с подкреплением (1)

Игра агента со средой:

- 1 инициализация стратегии $\pi_1(a | s)$ и состояния среды s_1
- 2 для всех $t = 1, \dots, T, \dots$
 - 1 агент выбирает действие $a_t \sim \pi_t(a | s_t)$;
 - 2 среда генерирует премию $r_{t+1} \sim p(r | a_t, s_t)$ и новое состояние $s_{t+1} \sim p(s | a_t, s_t)$;
 - 3 агент корректирует стратегию $\pi_t(a | s)$;

Полная постановка задачи обучения с подкреплением (2)

- Это марковский процесс принятия решений (MDP), если

$$\begin{aligned} P\{s_{t+1} = s', r_{t+1} = r \mid s_t, a_t, r_t, s_{t-1}, a_{t-1}, r_{t-1}, \dots, s_1, a_1\} \\ = P\{s_{t+1} = s', r_{t+1} = r \mid s_t, a_t\} \end{aligned}$$

- МППР называется **финитным**, если $|\mathcal{A}| < \infty$, $|\mathcal{S}| < \infty$.

Выгода - вознаграждение

- Суммарное вознаграждение (return) с момента t :

$$R_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots + r_{t+k} + \dots$$

- Сумма дисконтированных вознаграждений

$$R_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{k-1} r_{t+k} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

- дисконтирующий множитель $\gamma \in [0, 1]$ - оценка значений будущих вознаграждений
- моментальное вознаграждение важнее отложенных будущих:
 - $\gamma \rightarrow 0$ - близорукий агент
 - $\gamma \rightarrow 1$ - дальнзоркий агент

Функция ценности состояния и действия

- *Функция ценности состояния s при стратегии π :*

$$V^\pi(s) = \mathbb{E}_\pi [R_t \mid s_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right]$$

- *Функция ценности действия a в состоянии s при стратегии π :*

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_\pi [R_t \mid s_t = s, a_t = a] \\ &= \mathbb{E}_\pi [\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a] \end{aligned}$$

- E_{π} - мат.ожидание при условии, что агент следует стратегии π

Уравнение Беллмана (1)

- Пусть имеется полная информация о среде:
 $\mathcal{P}_{s,s'}^a = \Pr\{s_{t+1} = s' \mid s_t = s, a_t = a\}$ - вероятности переходов
 $\mathcal{R}_{s,s'}^a = \mathbb{E}\{r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'\}$ - ожидаемые премии
- Функцию полезности представим в виде двух слагаемых
 - немедленное вознаграждение r_{t+1}
 - дисконт-ое значение след. сост-я $\gamma V^\pi(s_{t+1})$

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi \{ R_t \mid s_t = s \} \\ &= \mathbb{E}_\pi [r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \mid s_t = s] \\ &= \mathbb{E}_\pi [r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \mid s_t = s] \\ &= \mathbb{E}_\pi [r_{t+1} \mid s_t = s] + \gamma \mathbb{E}_\pi [R_{t+1} \mid s_t = s]. \end{aligned}$$

Уравнение Беллмана (2)

- Пусть имеется полная информация о среде:
- $\mathcal{P}_{s,s'}^a = \Pr\{s_{t+1} = s' \mid s_t = s, a_t = a\}$ - вероятности переходов;
- $\mathcal{R}_{s,s'}^a = \mathbb{E}\{r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'\}$ - ожидаемые премии
- $\mathbb{E}_\pi [r_{t+1} \mid s_t = s] = \sum_a \sum_{s'} \mathcal{R}_{s,s'}^a \mathcal{P}_{s,s'}^a \pi(a \mid s)$
- $\mathbb{E}_\pi [R_{t+1} \mid s_t = s] = \sum_a \sum_{s'} \mathbb{E}_\pi [\sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_{t+1} = s'] \mathcal{P}_{s,s'}^a \pi(a \mid s)$
- $\mathbb{E}_\pi [\sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_{t+1} = s'] = V^\pi(s')$

Уравнение Беллмана (3)

- Система $|S|$ уравнений Беллмана с $|S|$ неизвестными функциями ценности состояния s - $V^\pi(s)$:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} \mathcal{P}_{s,s'}^a (\mathcal{R}_{s,s'}^a + \gamma V^\pi(s'))$$

- Аналогично с неизвестными функциями ценности действия a - $Q^\pi(s, a)$:

$$Q^\pi(s, a) = \sum_{s'} \mathcal{P}_{s,s'}^a (\mathcal{R}_{s,s'}^a + \gamma Q^\pi(s', a))$$

Уравнение Беллмана в матричном виде

Уравнение Беллмана в матричной форме:

$$V = \mathcal{R} + \gamma \mathcal{P}V,$$

где V - вектор-колонка с одной компонентой на состояние

$$\begin{bmatrix} V(1) \\ \dots \\ V(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}(1) \\ \dots \\ \mathcal{R}(n) \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \mathcal{P}_{1n} \\ \mathcal{P}_{n1} & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} V(1) \\ \dots \\ V(n) \end{bmatrix}$$

Решение уравнения Беллмана

- Уравнение Беллмана - это линейное уравнение
- Аналитическое решение

$$V = \mathcal{R} + \gamma \mathcal{P}V$$

$$(I - \gamma \mathcal{P})V = \mathcal{R}$$

$$V = (I - \gamma \mathcal{P})^{-1} \mathcal{R}$$

- Аналитическое решение только для небольших задач MDP
- Итерационные методы: дин.программирование, Монте-Карло и т.д.

Общий подход

- Хотелось бы иметь более надежную оценку $V(s)$ и $Q(s, a)$, приближающуюся к дисконт-ной выгоде R_t :

$$R_t^{(1)} = r_{t+1} + \gamma V(s_{t+1})$$

$$R_t^{(1)} = r_{t+1} + \gamma r_{t+2} + \gamma^2 V(s_{t+2})$$

...

$$R_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$$

$$R_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \dots$$

- Премии $r_{t+2}, r_{t+3}, \dots$ в момент t неизвестны, но можно усреднять прошлые, а не будущие наблюдения, и асимптотически это приводит к тому же результату!

Общий подход

- $s_1, a_1.r_1, \dots, s_k \sim \pi$

$$\begin{aligned} V_*(s) &= \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right] \\ &= \mathbb{E}_\pi \left[r_{t+1} + \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_t = s \right] \\ &= \mathbb{E}_\pi \left[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s \right] \\ &= \sum_{s'} \sum_r P(s', r \mid s, a) (r + \gamma V_*(s')) \end{aligned}$$

- Опыт $(s, a) \rightarrow (s', r)$ -> модель $P(s', r \mid s, a)$ -> решение ур-я Беллмана (indirect RL)
- Опыт $(s, a) \rightarrow (s', r)$ -> direct RL (без $P(s', r \mid s, a)$) -> π, V, Q -> действия

Монте-Карло подход к RL - оценка стратегии

- Цель: построить V^π по эпизодам взаимодействия по стратегии π

$$s_1, a_1.r_1, \dots, s_k \sim \pi$$

- Суммарное вознаграждение

$$R_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-1} r_T$$

- **Функция полезности**

$$V^\pi(s) = \mathbb{E}_\pi[R_t \mid s_t = s] \approx \frac{1}{N} \sum_{i=1}^N R_t^{(i)}$$

- Монте-Карло оценка стратегии использует **эмпирическое среднее** вместо **мат. ожидания**
- Обычный Монте-Карло оценивает ценность состояния как среднее арифметическое всех R_t , полученных после посещения этого состояния

Монте-Карло подход

- Монте-Карло оценка стратегии использует **эмпирическое среднее** вместо **мат. ожидания**
- Инкрементальный подход решает эту проблему.
- Вместо хранения всей истории мы храним только текущую оценку $V(s)$ и обновляем ее при каждом новом наблюдении

Монте-Карло оценка стратегии с первым посещением

- Оцениваем состояние s :
- для первого по времени посещения состояния s в эпизоде:
 - $N(s) \leftarrow N(s) + 1$
 - $S(s) \leftarrow S(s) + R_t$
- Полезность оцениваем как первую отдачу:
 $V(s) = N(s)/S(s)$
- По закону больших чисел

$$V(s) \xrightarrow{N(s) \rightarrow \infty} V^\pi(s)$$

- Смещенная состоятельная оценка с большой дисперсией

1. *Journal of the American Medical Association*, 2000; 283: 2689-2696.

$$\begin{aligned}\mu_k &= \frac{1}{k} \sum_{j=1}^k x_j = \frac{1}{k} \left(x_k + \sum_{j=1}^{k-1} x_j \right) \\ &= \frac{1}{k} (x_k + (k-1)\mu_{k-1}) \\ &= \mu_{k-1} + \frac{1}{k} (x_k - \mu_{k-1})\end{aligned}$$

Монте-Карло для приращений

- Обновим $V(s)$ с приращением для эпизода $s_1, a_1, r_1, \dots, s_T$
- Для каждого состояния s_t с отдачей R_t :

$$N(s_t) \leftarrow N(s_t) + 1$$

$$V(s_t) \leftarrow V(s_t) + \frac{1}{N(s_t)} (R_t - V(s_t))$$

- В нестационарных задачах можно отслеживать текущее среднее:

$$V(s_t) \leftarrow V(s_t) + \alpha (R_t - V(s_t))$$

- TD-обучение - это центральная идея в RL, объединяющая Монте-Карло и динамическое программирование. TD лежит в основе Q-learning, SARSA и современных deep RL методов.
- Методы временных разностей (temporal difference, TD) обучается напрямую по эпизодам взаимодействия со средой
- TD - безмодельный подход (model-free): модель переходов MDP и функция вознаграждения не известны
- TD обучается по эпизодам, используя бутстреп
- TD приближает значение на основе предыдущего приближения

Монте-Карло и TD

- TD может обучаться до того, как стала известна итоговая отдача
 - TD может обучаться интерактивно на каждом шаге
 - М-К должен дожидаться окончания эпизода, когда становится известной отдача
- TD может обучаться без информации об итоговой отдаче
 - TD может обучаться на неполных эпизодах
 - М-К может обучаться только на полных эпизодах
 - TD работает и для бесконечных окружений (без терминальных вершин)
 - М-К работает только в эпизодических окружениях (с терминальными вершинами)

Метод временных разностей TD

- TD учится на разнице между предсказаниями в разные моменты времени
- **Ключевая идея:** вместо ожидания конца эпизода и подсчета полной доходности (как Монте-Карло), можно обновлять оценку сразу, используя оценку следующего состояния как “предсказание будущего”
- Используется формула инкрементального Монте-Карло:
$$\text{Новая оценка} = \text{Старая оценка} + \text{Шаг} \times (\text{Цель} - \text{Старая оценка})$$

Метод временных разностей TD(0)

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right] \\ &= \mathbb{E}_\pi \left[r_{t+1} + \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \mid s_t = s \right] \\ &= \mathbb{E}_\pi \left[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s \right] \end{aligned}$$

- **Метод временных разностей:** После выбора a_t стали известны s_{t+1} и r_{t+1} , оцениваем $V^\pi(s_t)$ экспоненциальным скользящим средним:

$$V^\pi(s_t) := V(s_t) + \alpha_t (r_{t+1} + \gamma V(s_{t+1}) - V(s_t))$$

- $r_{t+1} + \gamma V(s_{t+1})$ - **TD-цель** (одношаговое предсказание)
- $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ - **TD-ошибка**

Метод временных разностей TD(0): алгоритм

- 1 Инициализировать $V(s)$ произвольно
- 2 Для каждого эпизода:
 - Инициализировать s
 - Пока s не терминальное:
 - Выбрать действие a по политике.
 - Выполнить a , получить r, s'
 - Обновить: $V(s) := V(s) + \alpha(r + \gamma V(s') - V(s))$
 - $s \leftarrow s'$

V обновляется сразу после каждого шага, а не в конце эпизода!

TD-ошибка как сигнал обучения

- TD-ошибка $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ - это мера “удивления” агента:
 - $\delta_t > 0$ реальность оказалась лучше ожиданий
 - $\delta_t < 0$ хуже ожиданий
 - $\delta_t \approx 0$ предсказание точное, обучение сошлось

Пример TD

- Сетка 4×4 . Агент стартует в S и должен дойти до G (цель)

S	.	.	.
.	X	.	.
.	.	X	.
.	.	.	G

- S - старт $(0, 0)$; G - цель $(3, 3)$, награда $+10$; X - ямы $(1, 1)$ и $(2, 2)$, награда -10 , эпизод завершается;
- Обычный шаг - награда -1 (чтобы агент искал короткий путь)
- Действия: вверх, вниз, влево, вправо
- Если агент упирается в стену — остается на месте

Пример TD

- Шаг 1: $s = (0, 0)$, $a = \rightarrow$, $r = -1$, $s' = (0, 1)$
 - $V(0, 1) = 0$
 - $V(0, 0) \leftarrow 0 + 0.1 \cdot (-1 + 0.95 \cdot 0 - 0) = -0.1$
- Шаг 2: $s = (0, 1)$, $a = \downarrow$, $r = -10$ (яма!), $s' = (1, 1)$ - терминал
 - $V(1, 1) = 0$ (терминал)
 - $V(0, 1) \leftarrow 0 + 0.1 \cdot (-10 + 0.95 \cdot 0 - 0) = -1.0$
- Эпизод завершен.

- Обновили $V(0, 0) = -0.1$ - старт “понял”, что шаг вправо стоит -1
- Обновили $V(0, 1) = -1.0$ - клетка перед ямой “поняла”, что идти вниз - катастрофа
- $V(0, 1)$ обновилось только со второго шага, потому что TD-цель использует V следующего состояния.
- Это и есть суть TD - ценность обновляется от терминала назад, но по одному шагу за раз.

Пример TD

- Допустим маршрут:
 $(0, 0) \rightarrow (1, 0) \rightarrow (2, 0) \rightarrow (2, 1) \rightarrow (2, 2)$
- Шаг 1: $s = (0, 0)$, $a = \downarrow$, $r = -1$, $s' = (1, 0)$
 - $V(1, 0) = 0$
 - $V(0, 0) \leftarrow -0.1 + 0.1 \cdot (-1 + 0.95 \cdot 0 - (-0.1)) = -0.19$
- Шаг 2: $s = (1, 0)$, $a = \downarrow$, $r = -1$, $s' = (2, 0)$
 - $V(2, 0) = 0$
 - $V(1, 0) \leftarrow 0 + 0.1 \cdot (-1 + 0 - 0) = -0.1$
- Шаг 3: $s = (2, 0)$, $a = \rightarrow$, $r = -1$, $s' = (2, 1)$
 - $V(2, 1) = 0$
 - $V(2, 0) \leftarrow -0.1 + 0.1 \cdot (-1 + 0 - (-0.1)) = -0.19$
- Шаг 4: $s = (1, 0)$, $a = \downarrow$, $r = -1$, $s' = (2, 0)$ $s = (2, 1)$,
 $a = \rightarrow$, $r = -10$ (яма!), $s' = (2, 2)$ - терминал
 - $V(2, 1) \leftarrow 0 + 0.1 \cdot (-10 + 0 - 0) = -1.0$

Пример TD

- Допустим на 5-м эпизоде агент случайно дошел до G .
Последние шаги: $\dots \rightarrow (3, 2) \rightarrow (3, 3)$
- Шаг k : $s = (3, 2)$, $a = \rightarrow$, $r = +10$, $s' = (3, 3)$ - терминал
 - $V(3, 3) = 0$
 - $V(3, 2) \leftarrow 0 + 0.1 \cdot (10 + 0 - 0) = +1.0$
- Теперь $V(3, 2)$ знает: “отсюда до цели - хорошо”. Но чтобы это знание дошло до старта, нужно много эпизодов

Пример TD

- Пример того, как выглядит обученная $V(s)$ после 200 эпизодов:

-3.1	-2.5	-1.6	-0.7
-2.2	-9.9	-1.2	-0.1
-1.4	-0.9	-9.9	9.5
-0.6	-0.1	9.0	10.0

- Видно, что ценность плавно растет по мере приближения к цели и резко падает у ям

Как из V получить маршрут

- Если известна модель переходов (мы знаем, куда ведет каждое действие), то выбираем действие, которое максимизирует $r + \gamma V(s')$:

$$\pi(s) = \arg \max_a [r(s, a) + \gamma V(s')]$$

- Для нашего примера из $(0, 0)$:
 - $\uparrow$: уперлись в стену, $r = -1$,
 $V(0, 0) = -3.1 \rightarrow -1 + 0.95 \cdot (-3.1) = -3.95$
 - $\downarrow$: $s' = (1, 0)$, $r = -1$,
 $V(1, 0) = -2.2 \rightarrow -1 + 0.95 \cdot (-2.2) = -3.09$
 - $\leftarrow$: стена $\rightarrow -3.95$
 - $\rightarrow$: $s' = (0, 1)$, $r = -1$,
 $V(0, 1) = -2.5 \rightarrow -1 + 0.95 \cdot (-2.5) = -3.38$
- Максимум — вниз (-3.09) . Агент пойдет вниз

- Не нужна модель среды
- Не нужно ждать конца эпизода
- работает в непрерывных и длинных задачах, где эпизоды могут длиться бесконечно.

TD-контроль

- SARSA и Q-learning - это TD-контроль: те же TD-обновления, но для $Q(s, a)$ и с целью, учитывающей следующее действие
- **SARSA (on-policy TD-контроль):**

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t (r_t + \gamma Q(s_{t+1}, a') - Q(s_t, a_t))$$

a' - действие, реально выбранное по текущей политике.

- Q-learning (off-policy TD-контроль):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)$$

Метод SARSA

- Рекуррентная формула для функции ценности $Q^\pi(s, a)$

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right] \\ &= \mathbb{E}_\pi \left[r_{t+1} + \gamma Q^\pi(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a \right] \end{aligned}$$

- Игра агента со средой:

- 1 иници-я стратегии $\pi_1(a \mid s)$ и состояния среды s_1 ;
- 2 для всех $t = 1, \dots, T, \dots$
- 3 агент выбирает действие $a_t \sim \pi_t(a \mid s_t)$, например, $a_t = \arg \max_{a \in A} Q(s_t, a)$ - **жадная** стратегия;
- 4 среда генерирует $r_t \sim p(r \mid s_t, a_t)$ и $s_{t+1} \sim p(s \mid s_t, a_t)$;
- 5 агент разыгрывает еще один шаг $a' \sim \pi_t(a \mid s_{t+1})$;
- 6 $Q(s_t, a_t) := Q(s_t, a_t) + \alpha_t (r_t + \gamma Q(s_{t+1}, a') - Q(s_t, a_t))$

Метод SARSA - пример

- Допустим, агент в $(0, 0)$, выбрал $\rightarrow$, попал в $(0, 1)$. Там ε -greedy выбрал $\downarrow$ (в яму)
- Обновление SARSA: $s = (0, 0)$, $a = \rightarrow$, $r = -1$, $s' = (0, 1)$, $a' = \downarrow$
 - $Q(0, 1, \downarrow) = 0$ (еще не знаем, что там яма)
 - $Q(0, 0, \rightarrow) \leftarrow 0 + 0.1 \cdot (-1 + 0.95 \cdot 0 - 0) = -0.1$
- Следующий шаг: $s = (0, 1)$, $a = \downarrow$, $r = -10$, $s' = (1, 1)$ - терминал
 - $Q(0, 1, \downarrow) \leftarrow 0 + 0.1 \cdot (-10 + 0 - 0) = -1.0$
- SARSA обновляет $Q(0, 0, \rightarrow)$ до того, как узнал, что $\downarrow$ ведет в яму. Поэтому его оценка “мягче”

- $$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$$

- $G_t^{(1)} = r_{t+1} + \gamma V(s_{t+1})$ - цель TD(0)
- $G_t^{(2)} = r_{t+1} + \gamma V(s_{t+1}) + \gamma^2 V(s_{t+2})$ - двухшаговая цель
- $G_t^{(\infty)} = G_t$ - цель Монте-Карло

Метод временных разностей TD(λ)

- $TD(\lambda)$ использует взвешенное среднее всех n -шаговых доходностей с геометрическими весами:

$$G_t^{(\lambda)} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- Вес каждой n -шаговой доходности: $(1 - \lambda)\lambda^{n-1}$
- Практическая реализация $TD(\lambda)$ использует следы пригодности (eligibility traces)
- **Идея:** каждое посещенное состояние оставляет след $E_t(s)$, который:
 - увеличивается на 1 при посещении
 - затухает со временем с коэффициентом $\gamma\lambda$

Метод временных разностей TD(λ)

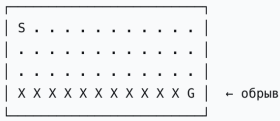
- **Идея:** каждое посещенное состояние оставляет след $E_t(s)$, который:
 - увеличивается на 1 при посещении
 - затухает со временем с коэффициентом $\gamma\lambda$
- Формула обновления следов:
$$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}(s_t = s)$$
- Обновление ценности: $V(s) \leftarrow V(s) + \alpha\delta_t E_t(s)$ для всех s
где $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ - TD-ошибка

TD(λ) - Аналогия

- Представьте, что вы ведете дневник своих действий
- Каждое действие оставляет чернильный след, который постепенно выцветает
- Когда делаете удачный ход, вы обновляете не только его, но и все недавние ходы, которые к нему привели (следы еще не выцвели)
- Чем свежее ход, тем ярче след, тем сильнее обновление
- Чем давнее — тем бледнее, тем слабее влияние
- Параметр λ управляет скоростью выцветания: $\lambda = 0$ - след исчезает мгновенно (TD(0)), $\lambda = 1$ - не исчезает никогда (Монте-Карло)

- **On-policy** - студент учится на своих же ошибках, решая задачи так, как умеет
- **Off-policy** - студент смотрит записи чужой игры (или своей старой) и учится играть лучше, чем играет сейчас
- **On-policy** обязан учиться на свежих данных, потому что политика меняется каждый шаг. Старый опыт «устаревает».
- **Off-policy** может хранить опыт в буфере (experience replay) и обучаться на нем много раз, даже если политика уже изменилась.

1. *Journal of the American Medical Association*, 2000; 284: 2689-2695.



- Старт S, цель G. Обычный шаг -1
- Вдоль нижнего края - обрыв X. Шаг туда → награда -100, возврат в старт
- Q-learning (**off-policy**) выучивает оптимальный путь - вдоль самого края обрыва. Это кратчайший маршрут, но если ϵ -greedy случайно шагнет вниз - катастрофа
- SARSA (**on-policy**) выучивает безопасный путь - на одну клетку выше от края. Он «знает», что политика иногда ошибается (ϵ -greedy), и учитывает это в оценке. Путь длиннее, но надежнее.

Жадные стратегии максимизации ценности

- Рекуррентная формула для функции ценности $Q^\pi(s, a)$:

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a \right] \\ &= \mathbb{E}_\pi \left[r_t + \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right] \\ &= \mathbb{E}_\pi \left[r_t + \gamma Q^\pi(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a \right] \end{aligned}$$

- Уравнение Беллмана для оптимальной функции ценности Q^* :

$$Q^*(s, a) = \mathbb{E}_\pi \left[r_t + \gamma \max_{a' \in A} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \right]$$

Жадные стратегии максимизации ценности

- Уравнение Беллмана для оптимальной функции ценности Q^* :

$$Q^*(s, a) = \mathbb{E}_\pi \left[r_t + \gamma \max_{a' \in A} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \right]$$

- Утв. Жадная стратегия “выбирать то действие, на котором достигается максимум в уравнениях Беллмана” оптимальна:

$$\pi_{t+1}(a) = \frac{1}{|A_t|} [a \in A_t], \quad A_t = \arg \max_{a \in A} Q^*(s_t, a)$$

Метод Q-обучения (1)

- Уравнение Беллмана $Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q(s', a')]$
- Аппроксимируем $Q^*(s, a)$ экспоненциальным скользящим средним (заменяем мат.ожидание средним по раундам):

$$Q(s_t, a_t) := Q(s_t, a_t) + \alpha_t \left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)$$

- инициализация стратегии $\pi_1(a | s)$ и состояния среды s_1 ;
- агент выбирает действие $a_t \sim \pi_t(a | s_t)$, например, $a_t = \arg \max_{a \in A} Q(s_t, a)$ - жадная стратегия;
- среда генерирует $r_t \sim p(r | s_t, a_t)$ и $s_{t+1} \sim p(s | s_t, a_t)$;
- $Q(s_t, a_t) := Q(s_t, a_t) + \alpha_t (r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$

Метод Q-обучения (2)

- Ключевая особенность: используется $\max$ по действиям, то есть алгоритм учится, как будто агент всегда выбирает лучшее действие - это off-policy метод

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t (r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$$

- **УТВ.** Если α_t уменьшается ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$), и все s посещаются бесконечное число раз, то $Q \rightarrow^{\text{пн}} Q^*$, $t \rightarrow \infty$

- Возможны два способа выбора действий:
- **on-policy**: $a_t \sim \pi_t(a|s_t) \Leftrightarrow a_t \in \arg \max_{a \in A} Q(s_t, a)$.
SARSA – on-policy TD-обучение. По каждому данному $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ обновить $Q(s_t, a_t)$.
- **off-policy**: $a_t \sim \pi_t(a|s_t)$ - другая стратегия на основе Q . Q-обучение – off-policy TD-обучение. Сразу решаем уравнения Беллмана относительно максимумов.

Метод Q-обучения (алгоритм)

- ❶ Инициализировать $Q(s, a)$ произвольно (часто нулями).
- ❷ Для каждого эпизода:
 - ❶ Инициализировать начальное состояние s
 - ❷ Пока эпизод не завершен:
 - ❶ Выбрать действие a по поведенческой политике (например, ϵ -жадную по Q)
 - ❷ Выполнить a , получить r, s'
 - ❸ Обновить:
$$Q(s, a) := Q(s, a) + \alpha (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$
 - ❹ Перейти: $s \leftarrow s'$

Метод Q-обучения (пример)

- Сетка 4×4 . Агент стартует в S и должен дойти до G (цель)

S	.	.	.
.	X	.	.
.	.	X	.
.	.	.	G

- S - старт $(0, 0)$; G - цель $(3, 3)$, награда $+10$; X - ямы $(1, 1)$ и $(2, 2)$, награда -10 , эпизод завершается;
- Обычный шаг - награда -1 (чтобы агент искал короткий путь)
- Действия: вверх, вниз, влево, вправо
- Если агент упирается в стену — остается на месте

Метод Q-обучения (пример)

- Агент ведет таблицу Q : $4 \times 4 = 16$ строк (по одной на каждую клетку) и 4 столбца (по одному на каждое действие).
- Изначально вся таблица заполнена нулями.

Клетка	↑	↓	←	→
(0, 0)	0	0	0	0
(0, 1)	0	0	0	0
...	...	...	...	...
(3, 3)	0	0	0	0

- Смысл числа $Q(\text{клетка}, \text{действие})$: «насколько хорошо пойти этим действием из этой клетки, если дальше действовать оптимально»

Метод Q-обучения (пример)

- Каждый раз, когда агент делает шаг, он пересчитывает одну клетку таблицы по формуле:
- Новое $Q = \text{Старое } Q + \alpha \times (\text{Награда} + \gamma \times \text{Лучшее } Q \text{ в новой клетке} - \text{Старое } Q)$
- Награда - что получили прямо сейчас
- Лучшее Q в новой клетке - максимум по 4 действиям в клетке, куда попали
- «Я думал, что это действие стоит X . Но реальность дала мне награду + будущую ценность. Сдвину свою оценку чуть-чуть в сторону reality»

Метод Q-обучения (пример - жадная стратегия)

- Как агент выбирает действие: ϵ -greedy
- С вероятностью ϵ (например, 0.2) агент делает случайный шаг - это исследование
- С вероятностью $1 - \epsilon$ (0.8) агент выбирает действие с максимальным Q - это эксплуатация
- Сначала, пока таблица нулевая, агент почти всегда блуждает. Постепенно Q -значения становятся информативными, и жадный выбор начинает вести к цели

Метод Q-обучения (пример - первый эпизод, шаг 1)

Шаг 1. Агент в $(0, 0)$. Все $Q = 0$, выбирает случайно, например, «вправо».

- Переходит в $(0, 1)$, получает награду -1 .
- Смотрит: лучшее Q в $(0, 1) = 0$ (таблица пустая).
- Обновляет
$$Q(0, 0, \rightarrow) = 0 + 0.1 \times (-1 + 0.95 \times 0 - 0) = -0.1.$$

Метод Q-обучения (пример - первый эпизод, шаг 2)

Шаг 2. Агент в $(0, 1)$. Случайно выбирает, например, «вниз».

- Переходит в $(1, 1)$, а там яма! Награда -10 , эпизод завершен.
- Обновляет $Q(0, 1, \downarrow) = 0 + 0.1 \times (-10 + 0 - 0) = -1.0$.
- Эпизод закончился. Агент уже узнал: «вниз из $(0, 1)$ плохо».

Метод Q-обучения (расползание от цели)

- Представим, что агент случайно дошел до цели. Скажем, из $(3, 2)$ шагнул «вправо» в $(3, 3)$
 - Награда $+10$, эпизод завершен
 - Лучшее Q в $(3, 3) = 0$ (терминальное состояние)
 - Обновляет $Q(3, 2, \rightarrow) = 0 + 0.1 \times (10 + 0 - 0) = +1.0$

Метод Q-обучения (расползание от цели)

- Теперь, когда агент снова окажется в $(3, 1)$ и шагнет «вправо» в :
 - Награда -1 , лучшее Q в $(3, 2)$ теперь $+1.0$ (это действие «вправо»).
 - $Q(3, 1, \rightarrow) = 0 + 0.1 \times (-1 + 0.95 \times 1.0 - 0) = -0.005$
- Уже почти 0 , а не -1 . Постепенно значения «просачиваются» назад: $(3, 0) \rightarrow (2, 0) \rightarrow \dots \rightarrow \text{старт}$. Это и есть temporal difference: ценность передается от цели к старту шаг за шагом, эпизод за эпизодом.

Почему ямы получают огромные отрицательные Q

- Как только агент шагнет в яму, он запишет в $Q(s, a)$ -10 (умноженное на α и γ -цепочку)
- Поскольку яма - это терминальное состояние, ценность не «размывается» будущими шагами
- Со временем Q -значения действий, ведущих в ямы, станут очень отрицательными. Жадная политика начнет их избегать

- После 1000 эпизодов таблица Q заполнена числами. Если в каждой клетке взять действие с максимальным Q , получится стрелочная карта:

$\rightarrow$	$\rightarrow$	$\rightarrow$	$\downarrow$
$\downarrow$	X	$\uparrow$	$\downarrow$
$\downarrow$	$\downarrow$	X	$\downarrow$
$\rightarrow$	$\rightarrow$	$\rightarrow$	G

Особенности Q-обучения

- Агент не знает карту мира. Он просто пробует действия, он не знает, где цель, не знает, где ямы. Он получает награды и после каждого шага чуть-чуть правит одну клетку таблицы Q в сторону «награда + лучшая будущая оценка». Он видит только «я в клетке, сделал шаг, получил награду».
- Постепенно, благодаря тому, что ценность цели расползается назад по цепочке состояний, вся таблица наполняется осмысленными числами.
- Жадный выбор по этой таблице дает оптимальный маршрут. И все это без модели среды, без подсказок и без нейросетей, только награды и повторение

Особенности Q-обучения

- Обучение идет маленькими шагами (например $\alpha = 0.1$)
- Каждое обновление сдвигает Q лишь на 10% от TD-ошибки:
 - Новое $Q = \text{Старое } Q + 0.1 \times (\text{цель} - \text{старое } Q)$
- Это значит, что одного посещения клетки недостаточно, чтобы Q -значение стало точным.

Адаптивный epsilon-жадный метод временных разностей

Идея: Чем сильнее колебания (дисперсия) $Q_t(s, a)$, тем больше должна быть вероятность ε_t исследовательских действий

$$f(s, a) = \left| \frac{\exp(Q_t(s, a)/\sigma) - \exp(Q_{t+1}(s, a)/\sigma)}{\exp(Q_t(s, a)/\sigma) + \exp(Q_{t+1}(s, a)/\sigma)} \right|$$

$$\varepsilon_{t+1}(s) = \varepsilon_t(s) + \delta (f(s_t, a_t) - \varepsilon_t(s))$$

Рекомендации: $\delta = 1/|A(s)|$, $A(s)$ - число возможных действий в состоянии s

σ - обратная чувствительность - $\sigma \rightarrow 0$ - чисто исследов. стратегия

инициализация $\varepsilon_t(s) = 1$ - чисто исследов. стратегия

Deep Q-learning

- Deep Q-Learning (**DQN**) - расширение Q-learning, применяемое в задачах с большими пространствами состояний, где таблица Q невозможна.
- Таблицу Q заменяет нейронная сеть, которая учится предсказывать ценность действий по “сырым” данным (например, пикселям экрана)
- Сеть в качестве аппроксиматора функции Q :
 - **Вход**: текущее состояние s (например, пиксели экрана)
 - **Выход**: вектор Q -значений для всех возможных действий a из этого состояния
 - **Обучение**: сеть обучается как задача регрессии, где “правильный ответ” (цель) вычисляется по уравнению Беллмана

Q-обучение

- Оптимальное значение Q удовлетворяет уравнению Беллмана

$$Q^*(s, a) = E_{s'} \left[r + \gamma \max_{a'} Q(s', a') \mid s, a \right]$$

- Используем правую часть $r + \gamma \max_{a'} Q(s', a', \mathbf{w})$ в качестве цели
- Минимизируем MSE-потери градиентным спуском

$$l = \left(r + \gamma \max_a Q(s', a', \mathbf{w}) - Q(s, a, \mathbf{w}) \right)^2$$

- Сходится к Q^*
- Но расходятся при использовании нейронных сетей из-за
 - корреляции между примерами
 - нестационарной цели

Два ключевых трюка для стабильности

- 1 **Experience Replay** (Воспроизведение опыта): Агент сохраняет свои переходы (s, a, r, s') в большой буфер памяти. Во время обучения он берет **случайные мини-батчи** из этого буфера, а не учится на последовательных шагах. Это разрывает корреляцию между данными и делает обучение более похожим на стандартное обучение с учителем
- 2 **Target Network** (Целевая сеть): Проблема в том, что цель Target зависит от той же сети, которую мы обучаем. Если сеть меняется, цель “убегает”. Решение - использовать **вторую замороженную сеть** для расчета цели. Веса основной сети периодически копируются в целевую сеть (например, раз в 1000 шагов)

Безмодельный и модельный подходы

- **Модель среды** - это описание того, как среда реагирует на действия агента. Формально она отвечает на два вопроса:
 - 1 Динамика переходов: если агент в состоянии s сделает действие a , в какое состояние s' он попадет и с какой вероятностью? $P(s' | s, a)$
 - 2 Функция награды: какую награду r он получит при этом переходе? $R(s, a, s')$
- Если агент **знает** эти две вещи - у него есть модель среды. Если **не знает** - он “безмодельный”.

Безмодельный и модельный подходы

- **Безмодельный подход (model-free)** - это класс алгоритмов RL, которые не строят и не используют модель среды. Они учатся непосредственно на опыте взаимодействия: пробуют действия, наблюдают награды и переходы, и на основе этого корректируют оценки ценности или политику
- **Модельный подход (model-based)** - алгоритмы, которые строят модель среды (или получают ее извне) и используют ее для планирования и обучения

Безмодельный и модельный подходы на примере

- **Безмодельный агент** в нашей сетке 4×4 :
 - не знает, где стены, не знает, где ямы, не знает, куда ведет каждое действие, просто шагает, получает r и s' , и обновляет Q-таблицу.
- **Модельный агент**:
 - сначала исследует среду и запоминает: “из $(0, 0)$ вправо $\rightarrow$, награда”,
 - строит внутреннюю карту переходов и наград,
 - затем планирует в уме: “если я пойду так, то...”, не трогая реальную среду.

Безмодельные и модельные алгоритмы

- **Безмодельные:** Q-learning, SARSA, $TD(\lambda)$, SARSA(λ), $Q(\lambda)$, DQN и все его варианты (Double, Dueling, Rainbow), Policy Gradient (REINFORCE), Actor-Critic (A2C, A3C, PPO, SAC, DDPG, TD3)
- **Модельные:** Dyna-Q (Q-learning + обучение модели), MCTS (Monte Carlo Tree Search) - используется в AlphaGo, PILCO, PETS, Dreamer, MuZero, Value Iteration, Policy Iteration (если модель известна)

Аналогии

- **Безмодельный агент** учится методом проб и ошибок как ребенок, который трогает горячее и запоминает, что это больно.
- **Модельный агент** сначала строит внутреннюю симуляцию мира как шахматист, который просчитывает ходы в голове, не двигая фигуры на доске.

Еще аналогии

- **Безмодельный агент** - это турист в незнакомом городе без карты. Он ходит, натывается на тупики, запоминает: “здесь тупик”, “здесь ресторан”. Постепенно учится ходить оптимально, но ценой многих ошибок.
- **Модельный агент** - турист, который сначала купил карту (или нарисовал ее сам, исследуя город). Теперь он может планировать маршрут в голове, не выходя из отеля, и проверять гипотезы мысленно.

Пример модельного агента (1)

- Сетка 4×4 . Агент стартует в S и должен дойти до G (цель)

S	.	.	.
.	X	.	.
.	.	X	.
.	.	.	G

- S - старт $(0, 0)$; G - цель $(3, 3)$, награда $+10$; X - ямы $(1, 1)$ и $(2, 2)$, награда -10 , эпизод завершается;
- Обычный шаг - награда -1 (чтобы агент искал короткий путь)
- Действия: вверх, вниз, влево, вправо
- Если агент упирается в стену — остается на месте

Пример модельного агента (2)

- Ключевое отличие от безмодельного агента: агент не знает заранее, где ямы и цель. Но он запоминает результаты своих действий и строит внутреннюю карту мира - модель переходов и наград
- Модельный агент хранит 2 таблицы: модели переходов $M(s, a) \rightarrow s'$ и наград $R(s, a) \rightarrow r$

	s	a	s'
$M(s, a)$	$(0, 0)$	$\rightarrow$	$(0, 1)$
	$(0, 0)$	$\downarrow$	$(1, 0)$
	$\dots$	$\dots$	$\dots$

	s	a	r
$R(s, a)$	$(0, 0)$	$\rightarrow$	-1
	$(1, 1)$	любое	-10
	$(3, 2)$	$\rightarrow$	$+10$
	$\dots$	$\dots$	$\dots$

Алгоритм Dyna-Q

- Действовать в реальной среде: выбрать a , получить r , s'
- Обновить Q по обычной формуле Q-learning.
- Обновить модель: запомнить $M(s, a) = s'$ и $R(s, a) = r$
- Планировать: повторить N раз:
 - взять случайное ранее посещенное состояние s
 - взять случайное действие a , которое там пробовали
 - “вспомнить” $s' = M(s, a)$ и $r = R(s, a)$
 - обновить $Q(s, a)$ как будто это был реальный шаг.

- $$(0, 0) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (1, 2) \rightarrow (2, 2)$$

- Шаг 1: $s = (0, 0)$, $a = \rightarrow$, $r = -1$, $s' = (0, 1)$
 - $Q(0, 0, \rightarrow) \leftarrow 0 + 0.1 \cdot (-1 + 0.95 \cdot 0 - 0) = -0.1$
 - Модель: $M(0, 0, \rightarrow) = (0, 1)$, $R(0, 0, \rightarrow) = -1$
- Планирование ($N = 5$): пока модель почти пуста, обновлять особо нечего, только $(0, 0, \rightarrow)$. Обновляем $Q(0, 0, \rightarrow)$ еще 5 раз с тем же результатом:
 $Q \approx -0.41$ (постепенно сдвигается к -1)

Алгоритм Dyna-Q (2)

- Шаг 2: $s = (0, 1)$, $a = \rightarrow$, $r = -1$, $s' = (0, 2)$
 - $Q(0, 1, \rightarrow) \leftarrow -0.1$
 - Модель: $M(0, 1, \rightarrow) = (0, 2)$, $R(0, 1, \rightarrow) = -1$
 - Планирование: обновляем $Q(0, 0, \rightarrow)$ и $Q(0, 1, \rightarrow)$, теперь модель знает два перехода.
- Шаг 3: $s = (0, 2)$, $a = \downarrow$, $r = -1$, $s' = (1, 2)$
 - $Q(0, 2, \downarrow) \leftarrow -0.1$
 - Модель пополняется.

Алгоритм Dyna-Q (3)

- Шаг 4: $s = (1, 2)$, $a = \downarrow$, $r = -1$, $s' = (2, 2)$ — яма, терминальное состояние, но награду получаем до терминала:
 - $Q(1, 2, \downarrow) \leftarrow 0 + 0.1 \cdot (-1 + 0.95 \cdot 0 - 0) = -0.1$ (пока не знаем, что там яма)
 - Модель: $M(1, 2, \downarrow) = (2, 2)$, $R(1, 2, \downarrow) = -1$
- Шаг 5: $s = (2, 2)$, $a = \downarrow$, $r = -10$, терминальное состояние
 - $Q(2, 2, \downarrow) \leftarrow 0 + 0.1 \cdot (-10 + 0 - 0) = -1.0$
 - Модель: $M(2, 2, \downarrow) = \text{терминал}$, $R(2, 2, \downarrow) = -10$
- Эпизод завершен.

Алгоритм Dyna-Q (4)

- За один эпизод агент:
 - запомнил 5 переходов
 - обновил Q для 5 пар (s, a)
 - благодаря планированию ($N = 5$) успел “прокрутить” эти переходы еще 25 раз мысленно
- Это значит, что Q -значения уже не “свежие нули”, они частично сошлись к своим истинным значениям

- **Реальный шаг** (как в Q-learning):
 - Агент физически делает действие a в среде. Среда реально переходит в s'
 - Агент реально получает награду r . Обновляет $Q(s, a)$.
- **Мысленный шаг** (в Dyna-Q):
 - Агент вспоминает состояние s (из тех, что уже посещал)
 - Агент выбирает действие a (например, случайно из тех, что пробовал в s)
 - Агент спрашивает модель: “что будет, если я сделаю a в s ?”
 - Модель отвечает из памяти: $s' = M(s, a)$, $r = R(s, a)$
 - Агент обновляет $Q(s, a)$ по этой воображаемой паре
- Никакого движения в реальной среде. Только память.

Откуда модель знает ответ

- Модель - это записанный опыт. Когда агент реально шагнул, он запоминал:
 - Реальный шаг: $(0, 0) \rightarrow (0, 1)$, награда -1
 - Запись в модель: $M(0, 0, \rightarrow) = (0, 1)$, $R(0, 0, \rightarrow) = -1$
- Теперь, когда агент “мысленно” делает $\rightarrow$ из $(0, 0)$, модель возвращает запись:
 - переход: $(0, 1)$, награда: -1

- **Безмодельный агент (Q-learning)** - новичок, который учится только на **реальных партиях**. Сыграл - получил результат - запомнил. Медленно, но надежно.
- **Модельный агент (Dyna-Q)** - шахматист, который после партии садится и **проигрывает ее в голове**:
 - “А что если бы я пошел конем?”, “А что если бы я защитил пешку?”
 - Он не играет новые партии, он анализирует старые с помощью внутренней модели (знания правил и позиций).
 - Это позволяет учиться **быстрее**, чем на одних только реальных партиях.

?